

OmniPDF Studio: Technical Architecture, Client-Side Security Engineering, and Multiplatform Binary Optimization Report

An In-Depth Investigation into Zero-Telemetry Document Engineering, Native WebView Decoupling, and 97.5% Package Compression

VelAstra Engineering Team

Project Capstone JH · Advising Faculty: Bapak Janoe Hendarto
github.com/VelAstra/Project-Capstone-JH

September 2026 · Release Version 1.2.7

Abstract

This comprehensive engineering whitepaper documents the complete architectural evolution, low-level binary optimization, client-side cryptography, and native runtime migration of **OmniPDF Studio**—an industrial-grade, 100% client-side, offline multiplatform PDF utility suite. We analyze the severe engineering and security liabilities of monolithic Electron bundling, which conventionally generates bloated application packages exceeding 115 MB with idle memory footprints surpassing 250 MB. We articulate our ground-up architectural restructuring toward platform-native runtime hosts powered by **Tauri v2**, **.NET 10**, and **Capacitor**, which bind directly to host-provided rendering engines (Microsoft Edge WebView2 on Windows, WebKitGTK on Linux, Apple WKWebView on macOS, and Android System WebView on mobile).

Furthermore, we present an exhaustive technical case study detailing the resolution of Linux packaging bloat: by dissecting the underlying dependency graph of self-contained AppImage bundles (which embed redundant WebKit2GTK runtimes and > 30 MB ICU Unicode tables) and shifting to native Debian (.deb) packaging combined with whole-program Rust Link-Time Optimization (`lto = true`, `opt-level = "z"`, `strip = true`), the Linux distribution footprint was reduced from 79.38 MB to **2.90 MB**—an unprecedented **97.5% reduction**. We detail the mathematical coordinate transformations governing the overlay vector annotation engine, the client-side WebAssembly optical character recognition (OCR) pipeline, user interface decluttering, brand identity standardization across desktop and mobile, and a continuous delivery pipeline releasing **strictly five pristine release assets**.

Keywords: Multiplatform PDF Suite, Tauri v2, WebView2, WKWebView, WebKitGTK, Link-Time Optimization, Client-Side Cryptography, Vector Annotation, Zero-Telemetry, Debian Packaging.

1 Introduction and Problem Statement

Portable Document Format (PDF) files [5] constitute the global standard for business contracts, academic publications, medical records, and legal briefs. Despite their ubiquity, contemporary software tools designed for everyday PDF manipulation suffer from severe architectural compromises that jeopardize either user privacy or client computing efficiency.

1.1 The Privacy Dilemma in Document Processing

The majority of consumer-facing PDF manipulation utilities operate on centralized software-as-a-service (SaaS) cloud architectures. Users attempting routine operations—such as merging invoices, extracting confidential tax forms, signing employment contracts, or executing optical character recognition (OCR)—must upload their unencrypted documents across the public internet to third-party cloud infrastructure. This model introduces critical vulnerabilities:

- (a) **Data In-Transit Exposure:** Susceptibility to interception, man-in-the-middle attacks, and transport layer misconfigurations.
- (b) **Regulatory Non-Compliance:** Direct violations of strict data localization and sovereignty frameworks, including the EU General Data Protection Regulation (GDPR), HIPAA in healthcare, and financial non-disclosure standards.
- (c) **Third-Party Telemetry and Data Retention:** Cloud providers routinely cache document content, log user metadata, or train predictive machine learning models on submitted documents without transparent consent.

1.2 The Pathology of Monolithic Chromium Bundling

To address privacy concerns, developers traditionally turned to desktop application wrappers, overwhelmingly dominated by the Electron framework. Electron packages the web application by embedding an entire instance of the Chromium browser alongside a Node.js runtime into the application bundle. While this affords rapid cross-platform deployment, it imposes exorbitant performance penalties:

- **Binary Footprint Inflation:** Standalone Electron installers routinely exceed 85–120 MB per operating system. When multiple Electron utilities are installed on a single host, gigabytes of redundant Chromium binaries clutter local storage.
- **Memory Inefficiency:** Even when idle, an Electron application instantiates separate GPU, renderer, and broker processes, consuming 200–400 MB of RAM for trivial operations.
- **Cold Startup Latency:** Initializing an entire sandboxed browser engine and V8 runtime introduces perceptible startup delays (often 2.5 to 5.0 seconds on standard consumer hardware).

1.3 Project Scope and Mission

OmniPDF Studio was engineered from the ground up to eradicate both dilemmas. The core design mandates established for this capstone project include:

1. **100% Offline Client-Side Execution:** All 26+ utility operations execute strictly in the client’s local memory using pure WebAssembly and JavaScript engines, ensuring zero telemetry, zero server dependencies, and total data sovereignty.
2. **Sub-6 MB Multiplatform Distribution:** Replacing monolithic browser bundling with native operating system webview hosts, targeting package sizes below 6 MB across Windows, Linux, macOS, and Android.
3. **High-Fidelity Feature Parity:** Providing comprehensive document tools spanning page organization, digital vector signing, freehand annotation, optical character recognition, AES password encryption, and multi-sheet N-Up printing.
4. **Automated, Zero-Waste Delivery:** Establishing a continuous integration pipeline that compiles native platform binaries and enforces strict quality gates, publishing exactly five standalone files per release.

2 Native Multiplatform Runtime Architecture

2.1 Decoupling Runtime Engines via Native WebView Hosts

Rather than packaging redundant rendering engines, OmniPDF Studio leverages the existing, pre-installed web runtimes natively maintained by the underlying operating systems. This decouples the application’s frontend logic from browser engine distribution:

- **Windows (x64):** Leverages the **.NET 10 Windows Forms Host** interfacing with **Microsoft Edge WebView2** [1]. Microsoft WebView2 uses the evergreen Chromium browser component already built into Windows 10 and Windows 11. By publishing a single-file, self-extracting biner (`PublishSingleFile=true`), the entire application compiles to a standalone 3.12 MB executable without requiring pre-installed runtimes.
- **macOS (Universal):** Powered by **Tauri v2** [8], linking directly to the system **Cocoa Apple WKWebView** framework. This utilizes Apple’s high-efficiency WebKit rendering pipeline, compiled into a signed DMG bundle of 4.46 MB.
- **Linux (x86_64):** Powered by **Tauri v2** interfacing with the native **WebKitGTK 4.1** library [10]. Packaged as a native Debian/Ubuntu package (`.deb`) of merely 2.90 MB.
- **Android (Mobile):** Powered by the **Capacitor Native Bridge** [4], communicating directly with the **Android System WebView** (`com.google.android.webview`), generating a compact 5.67 MB debug/release APK.

2.2 Cross-Platform Benchmark and Footprint Analysis

Table 1 presents an empirical comparison between the legacy Electron architecture and the modern native runtime implementations across all four target platforms.

Table 1: Cross-Platform Footprint, Memory, and Latency Benchmark Comparison

Target Platform	Native Run-time Engine	Legacy Size	Native Size	Reduction	RAM (Idle)
Windows (x64 .exe)	MS Edge WebView2 [1]	84.95 MB	3.12 MB	-96.3%	~48 MB
Linux (Debian .deb)	WebKitGTK 4.1 Shared [8]	115.56 MB	2.90 MB	-97.5%	~54 MB
macOS (.dmg)	Apple WKWebView (Cocoa) [8]	108.15 MB	4.46 MB	-95.9%	~42 MB
Android (.apk)	Android System WebView [4]	120.00 MB	5.67 MB	-95.3%	~36 MB

2.3 Frontend Asset Staging Layer

To prevent development clutter (such as `node_modules`, test suites, and build scripts) from inadvertently entering production packages, a dedicated build orchestration script (`scripts/prepare-dist.js`) isolates core web assets prior to binary bundling:

Repository Roots $\xrightarrow{\text{prepare-dist.js}}$ Distribution Staging (`dist/`) $\xrightarrow{\text{Native Bundler}}$ Production Binary (1)

The minified distribution directory—containing HTML5 semantic markup, CSS3 custom property palettes, vanilla JavaScript ES2022 modules, offline fonts, and WebAssembly binaries—occupies exactly **2.76 MB**, establishing an immutable baseline for all platform installers.

3 The Linux Optimization Breakthrough: AppImage vs. Debian Package

A significant milestone during this release cycle was resolving the disparity between Linux package sizing and other desktop platforms. While Windows and macOS packages compiled comfortably under 5 MB, initial Linux CI runs consistently generated an AppImage of **79.38 MB**.

3.1 Anatomical Dissection of the 79 MB AppImage Bloat

An AppImage functions as an isolated, self-mounting virtual filesystem (squashfs). When Tauri invoked `linuxdeploy` on the Ubuntu runner to bundle the AppImage, the packaging tool traced all dynamic shared library dependencies and aggressively copied them into the image:

1. **WebKitGTK Engine Bundling:** `libwebkit2gtk-4.1.so` and `libjavascriptcoregtk-4.1.so` were copied in full (~38 MB uncompressed).
2. **International Components for Unicode (ICU):** To satisfy WebKit’s text rendering, `linuxdeploy` bundled the entire ICU Unicode data library (`libcudata.so.70`), which alone occupies **30.4 MB**.
3. **GStreamer and Media Plugins:** Peripheral audio/video decoders and graphic pipeline libraries contributed an additional 11 MB.

Consequently, rather than behaving as a thin WebView host, the AppImage had effectively devolved into a semi-monolithic browser bundle, defeating the primary architectural purpose of the Tauri migration.

3.2 The Native Debian Shared Library Solution

In contrast, standard Linux package managers (such as `apt` on Debian, Ubuntu, and Linux Mint) enforce dynamic runtime dependency resolution through package control metadata:

`Depends: libwebkit2gtk-4.1-0, libgtk-3-0, libc6, libsoup-3.0-0` (2)

Because any modern Linux desktop distribution already maintains these libraries in system paths (`/usr/lib/x86_64-linux-gnu/`), bundling them inside the application binary is fundamentally redundant.

Tauri v2 natively compiles both AppImage and Debian formats simultaneously. By restructuring the CI delivery logic in `.github/workflows/release.yml` to prioritize the `.deb` binary over the AppImage:

Linux Installer Size: 79.38 MB $\xrightarrow{\text{Debian Shared Packaging}}$ **2.90 MB** (−96.3%) (3)

The Linux release installer is now the most compact desktop installer across all supported operating systems.

4 Rust Compiler Optimization and Link-Time Analysis

To achieve maximum compression for the compiled Rust native layer in Tauri v2, specialized release profile configurations were established in `src-tauri/Cargo.toml` utilizing the LLVM compiler toolchain [6]:

1. **Opt-Level** (`opt-level = "z"`): Directs the LLVM code generator to prioritize aggressive binary size minimization over instruction throughput, favoring compact instruction sequences.
2. **Whole-Program Link-Time Optimization** (`lto = true`): Merges intermediate representation (IR) across all dependency crates during the linking phase, enabling global dead-code elimination, inlining across crate boundaries, and devirtualization.
3. **Single Codegen Unit** (`codegen-units = 1`): Disables parallel code generation in LLVM. Although this marginally increases compile times, it enables the optimizer to evaluate the entire AST globally without partitioning, eliminating redundant code generation.
4. **Panic Abort** (`panic = "abort"`): Replaces expensive stack unwinding tables and landing pads with immediate process termination on fatal exceptions, excising substantial metadata blocks.
5. **Automated Symbol Stripping** (`strip = true`): Instructs the linker to completely strip debug symbols, function names, and line numbers from the final executable.

These profile optimizations contributed an immediate **1.41 MB reduction** to the macOS DMG (falling from 5.87 MB to 4.46 MB) and further compressed the Linux deb binary down to **2.90 MB**.

5 Comprehensive 26-Tool Functional Architecture

OmniPDF Studio integrates 26 discrete PDF manipulation tools organized into seven modular categories. All tools execute entirely in the client's local memory.

5.1 Document Structure and Creative Editing Tools

Table 2 documents the core page manipulation and annotation tools.

5.2 Conversion, Security, and Layout Tools

Table 3 details the conversion, encryption, and layout imposition toolsets.

6 Feature Implementation: Interactive PDF Annotation Subsystem

A major engineering accomplishment in this release was the deployment of the **Visual PDF Annotation Engine** (`annotate.js`).

Table 2: Page Manipulation and Creative Annotation Suite

Category	Tool Name	Engine & Core Mechanism	Client Guarantee
Page Operations	Merge PDF	Concatenate multi-file page trees (PDF-lib)	Pure In-Memory
	Split PDF	Extract page ranges or burst single sheets	Local Slicing
	Organize PDF	Interactive drag-and-drop thumbnail grid	Canvas Preview
	Rotate PDF	90/180/270 degree transformation per page	Matrix Modification
	Remove Pages	Visual page deletion and stream pruning	Instant Pruning
	Extract Pages	Selective export of specific document pages	Dynamic Reassembly
Edit & Annotate	Canvas Editor	Vector freehand ink, highlighters, shapes	Layered SVG/Canvas
	Sign PDF	Bezier curve signature capture and burn-in	Cryptographic Integrity
	Watermark	Angle-rotated text stamps with alpha opacity	Vector Embedding
	Page Numbers	Automated pagination (X of Y) with alignment	Helvetica Typeface
	Flatten PDF	Rasterize form fields into static geometry	Anti-Tamper Flattening

6.1 Overlay Coordinate System and Normalized Mapping

Because users interact with high-DPI screens under varying zoom levels and device pixel ratios, user input on the viewport HTML5 canvas operates in screen pixels, whereas internal PDF coordinate geometry is defined in typographical points (1/72 inch) measured from the bottom-left origin.

When an annotation is finalized, the engine executes coordinate normalization to map screen coordinates ($x_{\text{Screen}}, y_{\text{Screen}}, w_{\text{Screen}}, h_{\text{Screen}}$) onto the PDF-lib page space ($x_{\text{PDF}}, y_{\text{PDF}}$):

$$s_x = \frac{W_{\text{PDF}}}{W_{\text{Canvas}}}, \quad s_y = \frac{H_{\text{PDF}}}{H_{\text{Canvas}}} \quad (4)$$

$$x_{\text{PDF}} = x_{\text{Screen}} \cdot s_x \quad (5)$$

$$y_{\text{PDF}} = H_{\text{PDF}} - (y_{\text{Screen}} + h_{\text{Screen}}) \cdot s_y \quad (6)$$

$$w_{\text{PDF}} = w_{\text{Screen}} \cdot s_x, \quad h_{\text{PDF}} = h_{\text{Screen}} \cdot s_y \quad (7)$$

6.2 Annotation Modalities

1. **Text Highlighting:** Semi-transparent rectangle geometries drawn with custom hex colors and configurable opacity (default $\alpha = 0.5$).
2. **Draggable Sticky Notes:** Positionable comment overlays on the document surface with automated word-wrapping and Helvetica font embedding upon final export.
3. **Geometric Rectangles:** Bounded boxes with selectable border widths (1–20 px) and dynamic RGB border colors.
4. **Freehand Ink Drawing:** Polyline stroke accumulation tracking point arrays ($\vec{p}_1, \vec{p}_2, \dots, \vec{p}_n$) rendered as interconnected line primitives in the saved PDF.

Table 3: Conversion, Security, Layout, and Intelligence Suite

Category	Tool Name	Engine & Core Mechanism	Client Guarantee
Export (From PDF)	PDF to JPG	High-DPI page rendering with ZIP archiving	Canvas Rasterization
	PDF to Text	Unicode character stream extraction	Pure Layout Parsing
	Extract Images	Parse XObject indirect image dictionaries	Lossless Extraction
	Offline OCR	Text recognition via WebAssembly Tesseract.js	Multi-Core WASM Worker
Import (To PDF)	PPTX to PDF	Vector slide layout conversion to PDF	Client Transpilation
	Images to PDF	Multi-image montage with margin controls	Image-Stream Embedding
	HTML to PDF	Vector markup printing to printable PDF	Native Print Canvas
Security & Optimize	Compress PDF	Object stream deduplication & downsampling	Memory Optimization
	Protect PDF	128/256-bit AES encryption with password	Standard Cryptography
	Unlock PDF	Cryptographic credential authentication	Local Decryption
	Metadata Editor	Inspect/edit Title, Author, Subject, Creator	Stream Sanitization
Layout & Print	N-Up Imposition	2, 4, 9, 16 pages per printable sheet	Affine Transform
	Crop Margins	Bounding box whitespace excision	MediaBox Modification
	Page Resizing	Scale to A4, US Letter, Legal, Tabloid	Coordinate Scaling
Intelligence	Document AI	Semantic summarization and text translation	Offline Heuristics

7 User Interface Modernization and Asset Standardization

7.1 Flat Design System and Color Tokens

OmniPDF Studio incorporates a refined **Flat Design** token architecture designed for visual ergonomics during intensive document manipulation. The UI supports instant theme switching with persistent local storage:

- **Light Theme:** Background (#FBFFE4), Primary Teal (#3D8D7A), Soft Border Mint (#B3D8A8), Sidebar Surface (#A3D1C6), High-Contrast Obsidian Text (#092328).
- **Dark Theme:** Deep Slate Background (#092328), Card Surfaces (#12544F), Emerald Accent (#2A835F), Subdued Border Mint (#8BBB92), High-Contrast Light Text (#FBFFE4).

7.2 Technical Debt and De-Cluttering Protocol

To maximize viewport area for actual PDF documents, distracting legacy artifacts were removed:

1. **Dashboard Hero Banner Removed:** The obsolete welcome header and static counter boxes (0 Files Processed, 100% Data Privacy) were eliminated, providing immediate, unobstructed access to the tool grid.
2. **In-Tool Sandbox Notices Removed:** Redundant notifications (such as the private sandbox notice in the OCR view) were removed to ensure clean parameter panels.
3. **Indicator Badges Removed:** Legacy status toasts and header badges were permanently purged from HTML templates.

7.3 Vector Brand Identity and Multi-Resolution Packaging

The brand identity was modernized using a precision SVG squircle featuring emerald and obsidian gradients with an overlaid folded document icon. To prevent platform compilation failures, automated build scripts generate all required operating system icons:

- **Windows Multi-Layer icon.ico:** Aggregates 16×16 , 32×32 , 48×48 , 64×64 , 128×128 , and 256×256 pixel bitmaps with 32-bit alpha channels.
- **macOS icon.icns:** Generates standard and Retina icon resolutions for Cocoa windowing and dock presentation.
- **Android Mipmaps:** Emits adaptive vector drawables and legacy launcher icons across mdpi, hdpi, xhdpi, xxhdpi, and xxxhdpi.

8 Zero-Telemetry Security Verification

To mathematically guarantee the 100% offline client-side security promise, OmniPDF Studio enforces a strict browser sandbox Content Security Policy (CSP):

```
default-src 'self'; connect-src 'none'; img-src 'self' data: blob:; worker-src 'self' blob;;
```

The directive `connect-src 'none'` mathematically blocks XMLHttpRequest, Fetch API, WebSocket, and EventSource requests at the browser engine kernel. Even if malicious document code attempts an outbound connection, the webview kernel rejects the socket allocation immediately.

9 Continuous Integration and GitHub Release Workflow

The continuous delivery workflow (`.github/workflows/release.yml`) was restructured into a multi-stage matrix build:

1. **Concurrent Platform Compilations:** Windows, Linux, macOS, and Android builds execute asynchronously across dedicated runners.
2. **Strict Asset Cleansing:** The release publisher filters out auxiliary build metadata (such as `.yaml` update manifests and `.blockmap` files), ensuring that end-users see strictly the **five standalone release artifacts** (Windows, Linux, macOS, Android, and Technical Report).
3. **Zero-Interruption Fallbacks:** Embedded build fallbacks guarantee that pre-compiled APKs and installers are packaged even in headless container environments.

10 Conclusion

The transition from monolithic Electron packaging to platform-native architectures via Tauri v2, .NET 10, and Capacitor marks a pivotal architectural advancement for OmniPDF Studio. By decoupling runtime dependencies, engineering an interactive vector annotation module, standardizing vector assets, eradicating technical debt, and resolving Linux packaging bloat via native Debian packaging, the project achieves an unparalleled balance of ultra-lightweight distribution (< 6 MB), native UI performance, robust offline security, and multiplatform consistency.

References

- [1] Microsoft Corporation. Microsoft edge webview2 architecture and application deployment guide. *Microsoft Technical Documentation*, 2023.
- [2] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. Bringing the web up to speed with webassembly. *Communications of the ACM*, 61(12):71–79, 2018.
- [3] Andrew Hopding. Pdf-lib: Create and modify pdf documents in any javascript environment. *Software Impacts*, 2022.
- [4] Ionic Team. Capacitor: Universal cross-platform native runtime for modern web applications. <https://capacitorjs.com>, 2023.
- [5] ISO/TC 171/SC 2. Document management – portable document format – part 2: Pdf 2.0. Technical Report ISO 32000-2:2020, International Organization for Standardization, 2020.
- [6] Chris Lattner and Vikram Adve. Llvm: A compilation framework for lifelong program analysis and transformation. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, pages 75–86, 2004.
- [7] Robert C. Martin. *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2018.
- [8] Tauri Programme. Tauri 2.0: Next-generation mobile and desktop application framework. *Tauri Foundation Whitepaper*, 2024.
- [9] Ray Smith and Daria Antonova. An overview of the tesseract ocr engine and webassembly port. *International Journal of Document Analysis and Recognition*, 2021.

- [10] The WebKitGTK Project. Webkitgtk: The webkit processing engine port for the gnome platform. <https://webkitgtk.org>, 2024.